PYTHON FOR VIDEO GAMES WITH PYGAME · LESSON 3

Shooting - lists and fire rate

Goal: By the end you can keep many bullets in a list, move them, remove them when they leave the screen and limit the fire rate.
Before you start:

- Lesson 2 - the UFO that moves with the arrows.

To copy the code: when you copy from a PDF, many viewers drop the spaces at the start of each line, and in Python those spaces matter. Every step is also a `.py` file in the same folder as this guide: open it and copy from there.

Step by step

Shooting: lists and fire rate

Your UFO already moves; now it is time to shoot. You will keep the bullets in a list, because there can be many at once. You will shoot with the space bar, without firing them all at once. And you will move the bullets, and remove them when they leave the screen.

Step 2

First, set up what you need. Below the UFO image, load the bullet image, and the shot sound, shoot.wav. A sound is loaded with pygame.mixer.Sound, and it plays when you call play. Then, next to speed, create an empty list, bullets: that is where every bullet you fire will live, because there can be many at once. And a cooldown variable, which starts at zero, and will count how long until you can shoot again.

Your `game.py` now looks like this — highlighted lines are new or changed (file `step-1.py`):

```
import pygame

pygame.init()
screen = pygame.display.set_mode((800, 600))
pygame.display.set_caption("Most Wanted UFO Mini")
clock = pygame.time.Clock()

background = pygame.image.load("assets/background_4.png").convert()
player_image = pygame.image.load("assets/player_1.png").convert_alpha()
bullet_image = pygame.image.load("assets/Bullet.gif").convert_alpha()
shoot_sound = pygame.mixer.Sound("assets/shoot.wav")

player = player_image.get_rect(midbottom=(400, 590))
speed = 6
bullets = []
cooldown = 0

running = True
while running:
```

```
    for event in pygame.event.get():
        if event.type == pygame.QUIT:
            running = False

    keys = pygame.key.get_pressed()
    if keys[pygame.K_LEFT]:
        player.x -= speed
    if keys[pygame.K_RIGHT]:
        player.x += speed
    player.clamp_ip(screen.get_rect())

    screen.blit(background, (0, 0))
    screen.blit(player_image, player)
    pygame.display.flip()
    clock.tick(60)

pygame.quit()
```

Step 3

Now the shot, inside the loop, after moving the UFO. Each pass of the loop is a frame, sixty per second: if cooldown is greater than zero, you subtract one. When the space bar is held, and cooldown has reached zero, you shoot. Shooting means adding a new Rect to the list with append, right above the UFO; then the sound plays, and cooldown goes back to fifteen. Sixty frames, split into waits of fifteen, give at most four shots per second. And to see them, after drawing the background, you loop over the list with a for loop, and draw each bullet with blit. They come after the background, so they sit on top.

Your `game.py` now looks like this — highlighted lines are new or changed (file `step-2.py`):

```
import pygame

pygame.init()
screen = pygame.display.set_mode((800, 600))
pygame.display.set_caption("Most Wanted UFO Mini")
clock = pygame.time.Clock()

background = pygame.image.load("assets/background_4.png").convert()
player_image = pygame.image.load("assets/player_1.png").convert_alpha()
bullet_image = pygame.image.load("assets/Bullet.gif").convert_alpha()
shoot_sound = pygame.mixer.Sound("assets/shoot.wav")

player = player_image.get_rect(midbottom=(400, 590))
speed = 6
bullets = []
cooldown = 0

running = True
while running:
    for event in pygame.event.get():
        if event.type == pygame.QUIT:
            running = False

    keys = pygame.key.get_pressed()
    if keys[pygame.K_LEFT]:
        player.x -= speed
    if keys[pygame.K_RIGHT]:
```

```
        player.x += speed
    player.clamp_ip(screen.get_rect())

    if cooldown > 0:
        cooldown -= 1
    if keys[pygame.K_SPACE] and cooldown == 0:
        bullets.append(bullet_image.get_rect(midbottom=player.midtop))
        shoot_sound.play()
        cooldown = 15

    screen.blit(background, (0, 0))
    for bullet in bullets:
        screen.blit(bullet_image, bullet)
    screen.blit(player_image, player)
    pygame.display.flip()
    clock.tick(60)

pygame.quit()
```

Step 4

Run the game, and hold the space bar while you move. The bullets appear, and each shot makes a sound, but they stay right where they spawned. Notice they come out spaced apart: that is the cooldown. One thing is missing: moving them.

Try it:

```
python game.py
```

Step 5

Now, right below the shot, loop over the list with a for loop, and every frame move each bullet ten pixels up. Remember: y grows downward, so subtracting from it means going up. Since this happens sixty times per second, the bullets fly up quickly and smoothly.

Your `game.py` now looks like this — highlighted lines are new or changed (file `step-3.py`):

```
import pygame

pygame.init()
screen = pygame.display.set_mode((800, 600))
pygame.display.set_caption("Most Wanted UFO Mini")
clock = pygame.time.Clock()

background = pygame.image.load("assets/background_4.png").convert()
player_image = pygame.image.load("assets/player_1.png").convert_alpha()
bullet_image = pygame.image.load("assets/Bullet.gif").convert_alpha()
shoot_sound = pygame.mixer.Sound("assets/shoot.wav")

player = player_image.get_rect(midbottom=(400, 590))
speed = 6
bullets = []
cooldown = 0

running = True
while running:
    for event in pygame.event.get():
```

```
        if event.type == pygame.QUIT:
            running = False

    keys = pygame.key.get_pressed()
    if keys[pygame.K_LEFT]:
        player.x -= speed
    if keys[pygame.K_RIGHT]:
        player.x += speed
    player.clamp_ip(screen.get_rect())

    if cooldown > 0:
        cooldown -= 1
    if keys[pygame.K_SPACE] and cooldown == 0:
        bullets.append(bullet_image.get_rect(midbottom=player.midtop))
        shoot_sound.play()
        cooldown = 15

    for bullet in bullets:
        bullet.y -= 10

    screen.blit(background, (0, 0))
    for bullet in bullets:
        screen.blit(bullet_image, bullet)
    screen.blit(player_image, player)
    pygame.display.flip()
    clock.tick(60)

pygame.quit()
```

Step 6

To remove them, you use a list comprehension: a for and an if in one line, building a new list. That new list has only the bullets whose bottom edge is still greater than zero; that is, the ones that have not left through the top. And you store it back in bullets, replacing the old one. If you do not remove them, the list grows forever, and the game slows down.

Your `game.py` now looks like this — highlighted lines are new or changed (file `step-4.py`):

```
import pygame

pygame.init()
screen = pygame.display.set_mode((800, 600))
pygame.display.set_caption("Most Wanted UFO Mini")
clock = pygame.time.Clock()

background = pygame.image.load("assets/background_4.png").convert()
player_image = pygame.image.load("assets/player_1.png").convert_alpha()
bullet_image = pygame.image.load("assets/Bullet.gif").convert_alpha()
shoot_sound = pygame.mixer.Sound("assets/shoot.wav")

player = player_image.get_rect(midbottom=(400, 590))
speed = 6
bullets = []
cooldown = 0

running = True
while running:
    for event in pygame.event.get():
```

```
        if event.type == pygame.QUIT:
            running = False

    keys = pygame.key.get_pressed()
    if keys[pygame.K_LEFT]:
        player.x -= speed
    if keys[pygame.K_RIGHT]:
        player.x += speed
    player.clamp_ip(screen.get_rect())

    if cooldown > 0:
        cooldown -= 1
    if keys[pygame.K_SPACE] and cooldown == 0:
        bullets.append(bullet_image.get_rect(midbottom=player.midtop))
        shoot_sound.play()
        cooldown = 15

    for bullet in bullets:
        bullet.y -= 10
    bullets = [b for b in bullets if b.bottom > 0]

    screen.blit(background, (0, 0))
    for bullet in bullets:
        screen.blit(bullet_image, bullet)
    screen.blit(player_image, player)
    pygame.display.flip()
    clock.tick(60)

pygame.quit()
```

Step 7

Run it again. Now the bullets fly up, and disappear when they leave through the top. Even if you shoot nonstop, you never see more than four or five bullets at once: the rest are already gone.

Try it:

```
python game.py
```

What you learned

Let's review. A list holds every bullet. Shooting means adding a Rect to that list. cooldown limits how many shots come out per second. And whatever leaves the screen is removed, so the list does not grow forever. Your exercise: make the UFO fire two bullets at once, one on each side. In the next lesson, the enemies arrive.

- A list holds many bullets
- Shooting is adding to the list
- cooldown limits the fire rate
- Remove what leaves the screen

Summary

- `bullets = []` creates an empty list; `bullets.append(...)` adds a bullet.

- Each bullet is a `Rect`: `bullet_image.get_rect(midbottom=player.midtop)` places it right above the UFO.
- `cooldown` counts frames: you can only shoot when it reaches 0, and shooting sets it back to 15 (4 shots per second at 60 FPS).
- `bullet.y -= 10` moves the bullet up; in pygame `y` grows downward.
- `bullets = [b for b in bullets if b.bottom > 0]` keeps only the bullets still on screen.
- `pygame.mixer.Sound("assets/shoot.wav")` loads a sound and `.play()` plays it.

Exercise

Start from this lesson's `game.py`.

- Make each shot add **two** bullets: one at `player.topleft` and one at `player.topright` (use `midbottom=` with those positions).
- Try different `cooldown` values. Which one feels best?
- Bonus: print `len(bullets)` to the terminal every frame and check that the list does not grow forever.

Solution

```
if keys[pygame.K_SPACE] and cooldown == 0:
    bullets.append(bullet_image.get_rect(midbottom=player.topleft))
    bullets.append(bullet_image.get_rect(midbottom=player.topright))
    shoot_sound.play()
    cooldown = 15
```

Art and sounds: Most Wanted UFO Classic, by Jose Daniel Soto, licensed CC BY 4.0.
A Magic Lab Studio course · magiclabstudio.com